信息科学技术学院

课程(实习）设计

课 程 设 计 名 称： 网络工程与组网实习

专　　业： 计算机科学与技术

学　　号： 2351610105

学 生 姓 名： 方泽宇

成　　绩：

批 改 日 期：

教 师 签 名：

目录

第一章 网络环境配置概述..... 1

第二章 网络配置过程、结果及测试..... 2

1. 网络仿真软件 Cisco Packet Tracer 5.3.0.0088 介绍..... 2

2. 实验环境..... 2

3. 配置过程(题目 1：二层交换机配置)..... 2

4. 结果及测试(题目 1：二层交换机配置)..... 4

5. 配置过程(题目 2：VLAN 配置)..... 6

6. 结果及测试(题目 2：VLAN 配置)..... 9

第三章 网络程序设计系统分析..... 12

项目要求..... 12

1. 需求分析..... 12

2. 可行性分析..... 13

3. 系统界面设计..... 错误!未定义书签。

4. 开发工具介绍..... 23

第四章 总结..... 26

第五章 工作心得与体会..... 27

第一章 网络环境配置概述

Windows11(25H2), Intel Core i7-12700H, 32GB-DDR5-4800MT/s, RTX3050Ti-GB, Cisco Packet Tracer 5.3.0.0088

第二章 网络配置过程、结果及测试

1. 网络仿真软件 Cisco Packet Tracer 5.3.0.0088 介绍

Cisco Packet Tracer 5.3.0.0088 是一款由思科（Cisco）开发的、用于学习和设计计算机网络的可视化网络仿真软件，它允许用户在图形界面中拖放构建网络拓扑，模拟思科路由器、交换机等设备，并使用命令行界面（CLI）进行配置与故障排除，非常适合网络初学者学习网络基础、配置和数据包传输过程，是 CCNA 等认证学习的得力工具。

主要特点

可视化操作：采用拖放式界面，方便添加、连接和管理网络设备，如路由器、交换机、PC 等。

真实模拟：可以模拟真实的网络环境和设备行为，包括数据包的收发、协议工作过程。

命令行接口（CLI）：支持配置思科设备的 CLI，让用户学习真实设备操作。

多平台支持：可在 Windows、Linux、macOS 等操作系统上运行。

学习辅助：是学习网络基础、思科认证（如 CCNA）的理想平台。

轻量级：相比真实设备，资源占用小，便于在普通计算机上进行实验。

版本特点

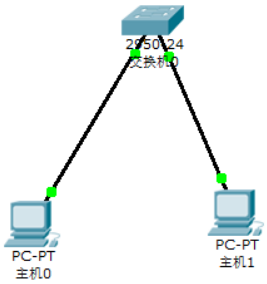
Cisco Packet Tracer 5.3 是一个非常强大的工具，提供了一个低成本、高效率的虚拟环境，让用户在安全、可控的条件下体验和掌握计算机网络技术。

2. 实验环境

Windows11(25H2)，Intel Core i7-12700H，32GB-DDR5-4800MT/s，RTX3050Ti-GB，Cisco Packet Tracer 5.3.0.0088

3. 配置过程(题目 1：二层交换机配置)

交换机的线路连接如下图所示：



打开交换机的命令行，执行以下操作：

1. 从用户模式进入特权模式：enable

```
Switch>enable
```

2. 从特权模式进入全局配置模式：configure terminal

```
Switch#configure terminal
Enter configuration commands, one per line.  End with CNTL/Z.
```

3. 设置叫混迹名称为 SwitchA：hostname SwitchA

```
Switch(config)#hostname SwitchA
```

4. 退到上层操作模式：exit

```
SwitchA(config)#exit
SwitchA#
%SYS-5-CONFIG_I: Configured from console by console
```

5. 从特权模式进入全局配置模式：configure terminal

```
SwitchA#configure terminal
Enter configuration commands, one per line.  End with CNTL/Z.
SwitchA(config)#
```

6. 禁止名称解析服务：no ip domain-lookup

```
SwitchA(config)#no ip domain-lookup
```

7. 退到上一级操作模式

```
SwitchA(config)#end
SwitchA#
%SYS-5-CONFIG_I: Configured from console by console
```

8. 配置管理 IP

```
SwitchA(config)#interface vlan 1
SwitchA(config-if)#ip address 192.168.0.1 255.255.255.0
SwitchA(config-if)#no shutdown

%LINK-5-CHANGED: Interface Vlan1, changed state to up

%LINEPROTO-5-UPDOWN: Line protocol on Interface Vlan1, changed state to up

SwitchA(config-if)#exit
```

9. 设置默认网关地址

```
SwitchA(config)#ip default-gateway 192.168.0.254
SwitchA(config)#emd
                 ^
% Invalid input detected at '^' marker.

SwitchA(config)#end
SwitchA#
%SYS-5-CONFIG_I: Configured from console by console
```

10. 从特权模式进入全局配置模式

```
SwitchA#configure terminal
Enter configuration commands, one per line.  End with CNTL/Z.
```

11. 进入端口 1 配置模式

```
SwitchA(config)#int f0/1
```

12. 配置端口速度

```
SwitchA(config-if)#speed ?
  10    Force 10 Mbps operation
  100   Force 100 Mbps operation
  auto  Enable AUTO speed configuration
SwitchA(config-if)#speed auto
```

13. 端口工作模式配置

```
SwitchA(config-if)#duplex ?
  auto  Enable AUTO duplex configuration
  full  Force full duplex operation
  half  Force half-duplex operation
SwitchA(config-if)#duplex duplex auto
                         ^
% Invalid input detected at '^' marker.

SwitchA(config-if)#duplex auto
SwitchA(config-if)#end
SwitchA#
%SYS-5-CONFIG_I: Configured from console by console
```

```
SwitchA#show run
Building configuration...

Current configuration : 1016 bytes
!
version 12.1
no service timestamps log datetime msec
no service timestamps debug datetime msec
no service password-encryption
!
hostname SwitchA
!
no ip domain-lookup
!
------------------此处省略----------------
---
!
interface FastEthernet0/24
!
interface Vlan1
 ip address 192.168.0.1 255.255.255.0
!
ip default-gateway 192.168.0.254
!
!
line con 0
!
line vty 0 4
 login
line vty 5 15
 login
!
!
end
```

14. 显示运行时配置文件内容(由于输出内容过长，因此直接拷贝配置内容如下)
15. 显示启动配置文件内容

```
SwitchA#show startup
startup-config is not present
```

4. 结果及测试(题目 1：二层交换机配置)

1. 显示 VLAN 信息

```
SwitchA#show interfaces vlan 1
Vlan1 is up, line protocol is up
  Hardware is CPU Interface, address is 0002.17ba.cc0b (bia 0002.17ba.cc0b)
  Internet address is 192.168.0.1/24
  MTU 1500 bytes, BW 100000 Kbit, DLY 1000000 usec,
     reliability 255/255, txload 1/255, rxload 1/255
  Encapsulation ARPA, loopback not set
  ARP type: ARPA, ARP Timeout 04:00:00
  Last input 21:40:21, output never, output hang never
  Last clearing of "show interface" counters never
  Input queue: 0/75/0/0 (size/max/drops/flushes); Total output drops: 0
  Queueing strategy: fifo
  Output queue: 0/40 (size/max)
  5 minute input rate 0 bits/sec, 0 packets/sec
  5 minute output rate 0 bits/sec, 0 packets/sec
     1682 packets input, 530955 bytes, 0 no buffer
     Received 0 broadcasts (0 IP multicast)
     0 runts, 0 giants, 0 throttles
     0 input errors, 0 CRC, 0 frame, 0 overrun, 0 ignored
     563859 packets output, 0 bytes, 0 underruns
     0 output errors, 23 interface resets
     0 output buffer failures, 0 output buffers swapped out
```

2. 显示端口信息(输出内容非常长，这里仅截取一小部分)



3. 在完成交换机配置后，将主机 0 的 IP 地址设置为 192.168.0.2，将主机 1 的 IP 地址设置为 192.168.0.3。随后在主机 0 中输入 ping 192.168.0.3，得到结果如下：

```
PC>ping 192.168.0.3

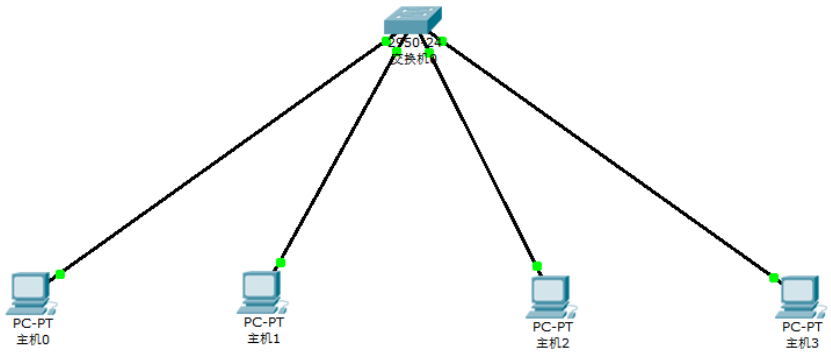
Pinging 192.168.0.3 with 32 bytes of data:

Reply from 192.168.0.3: bytes=32 time=104ms TTL=128
Reply from 192.168.0.3: bytes=32 time=63ms TTL=128
Reply from 192.168.0.3: bytes=32 time=62ms TTL=128
Reply from 192.168.0.3: bytes=32 time=62ms TTL=128

Ping statistics for 192.168.0.3:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 62ms, Maximum = 104ms, Average = 72ms
```

5. 配置过程(题目 2：VLAN 配置)

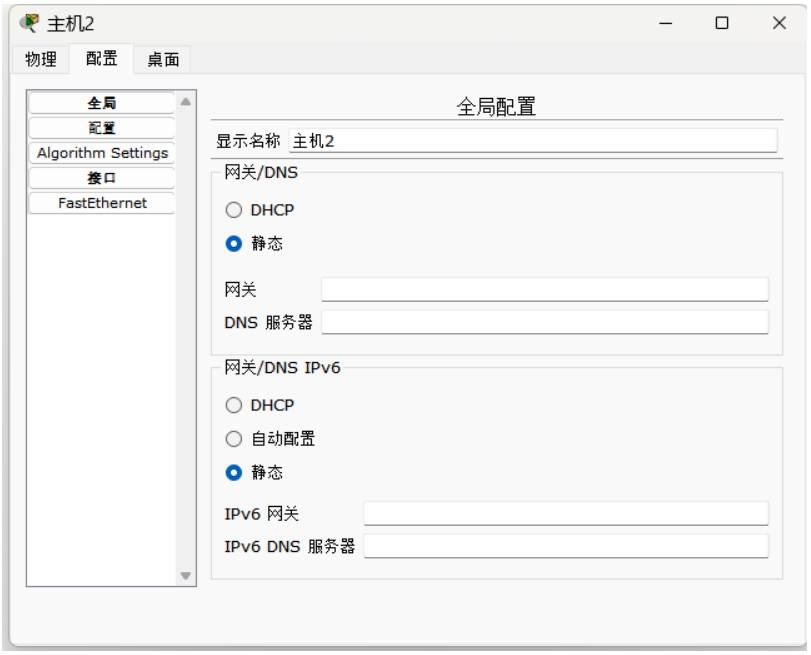
网络设备连线的拓扑图：



终端主机的配置：

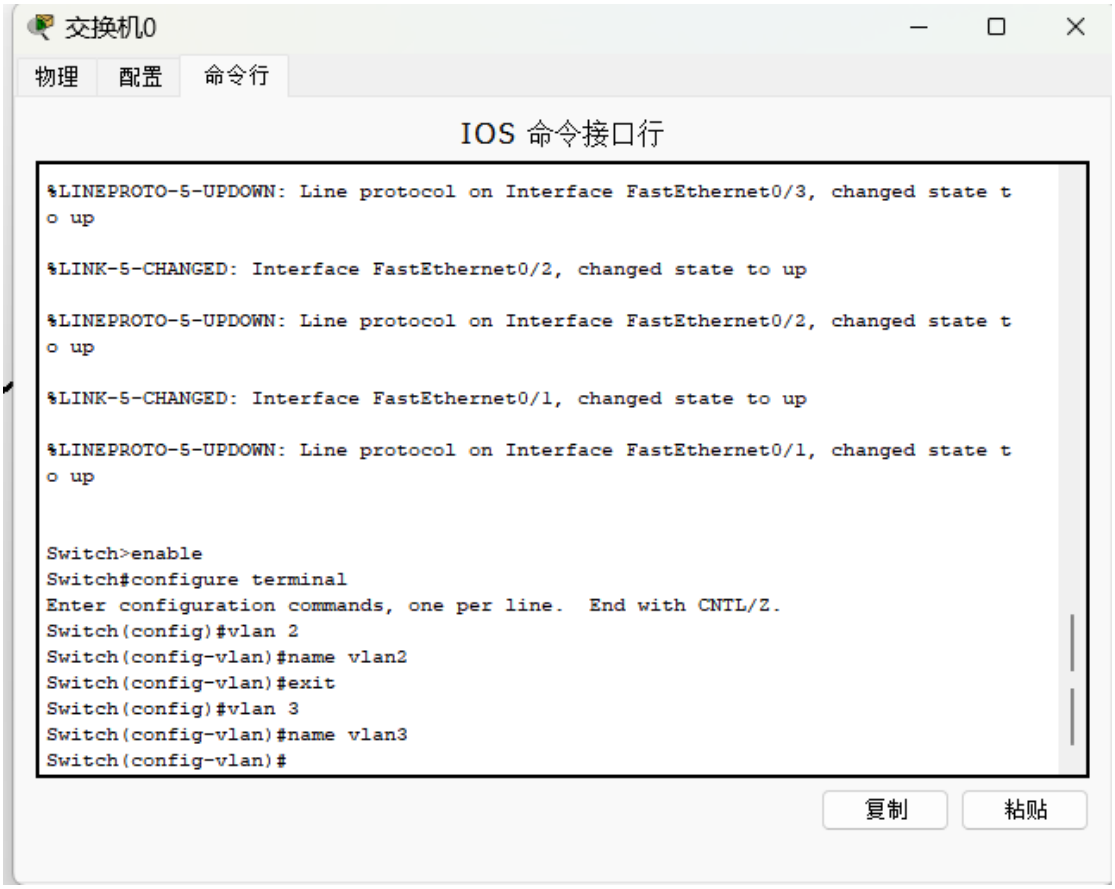



主机1
物理
配置
桌面
IP配置
DHCP
静态
IP 地址
192.168.1.2
子网掩码
255.255.255.0
默认网关
192.168.1.2
DNS 服务器
http:
WEB 浏览
Cisco IP Communicator
E Mail
PPPoE Dialer
Text Editor


主机2
物理
配置
桌面
全局
配置
Algorithm Settings
接口
FastEthernet
全局配置
显示名称
主机2
网关/DNS
DHCP
静态
网关
DNS 服务器
网关/DNS IPv6
DHCP
自动配置
静态
IPv6 网关
IPv6 DNS 服务器



划分 VLAN：创建 VLAN2 和 VLAN3

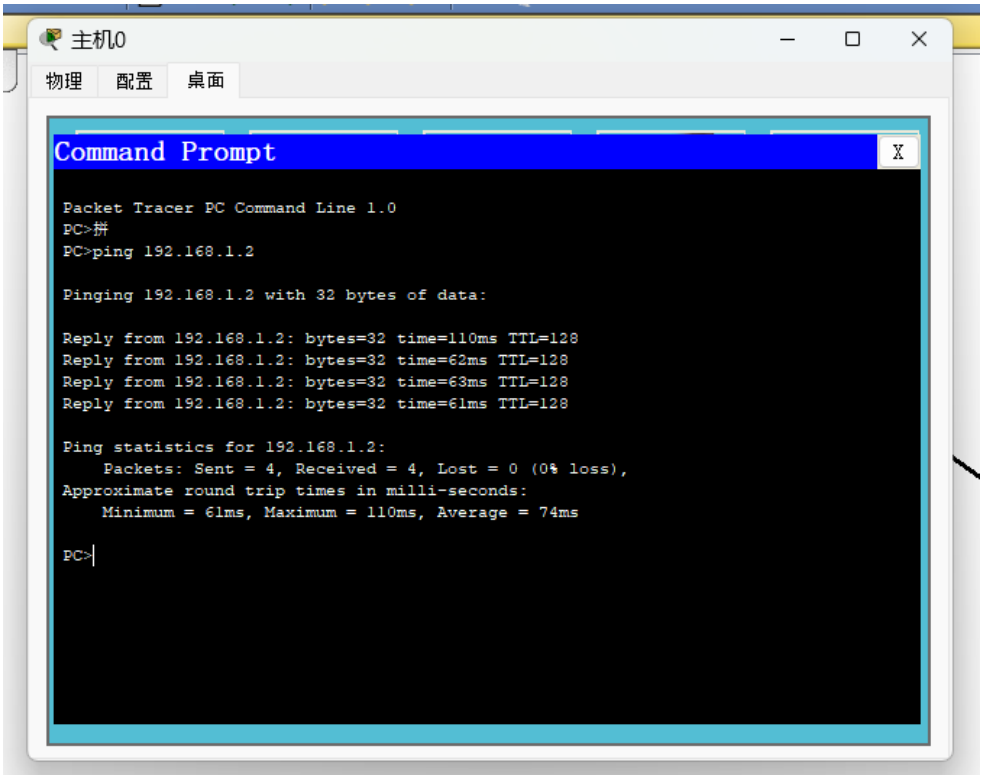


静态分配 VLAN 成员



6. 结果及测试(题目 2：VLAN 配置)

从 PC0 到 PC1 的测试



从 PC0 到 PC2 的测试：

```
PC>ping 192.168.1.3

Pinging 192.168.1.3 with 32 bytes of data:

Reply from 192.168.1.3: bytes=32 time=111ms TTL=128
Reply from 192.168.1.3: bytes=32 time=62ms TTL=128
Reply from 192.168.1.3: bytes=32 time=63ms TTL=128
Reply from 192.168.1.3: bytes=32 time=61ms TTL=128

Ping statistics for 192.168.1.3:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 61ms, Maximum = 111ms, Average = 74ms

PC>
```

从 PC0 到 PC3 的测试：

```
PC>ping 192.168.2.1

Pinging 192.168.2.1 with 32 bytes of data:

Request timed out.
Request timed out.
Request timed out.
Request timed out.

Ping statistics for 192.168.2.1:
    Packets: Sent = 4, Received = 0, Lost = 4 (100% loss),
```

测试划分 VLAN 后工作站 PC 机的连通性：

PC0 到 PC1：

```
^C
PC>ping 192.168.1.2

Pinging 192.168.1.2 with 32 bytes of data:

Reply from 192.168.1.2: bytes=32 time=117ms TTL=128
Reply from 192.168.1.2: bytes=32 time=62ms TTL=128
Reply from 192.168.1.2: bytes=32 time=61ms TTL=128
Reply from 192.168.1.2: bytes=32 time=62ms TTL=128

Ping statistics for 192.168.1.2:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 61ms, Maximum = 117ms, Average = 75ms

PC>
```

PC0 到 PC2：

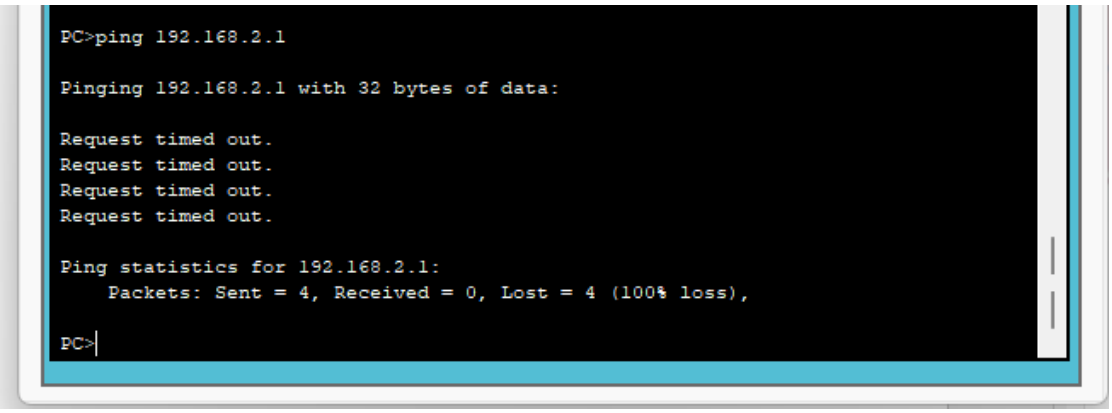
```
PC>ping 192.168.1.3

Pinging 192.168.1.3 with 32 bytes of data:

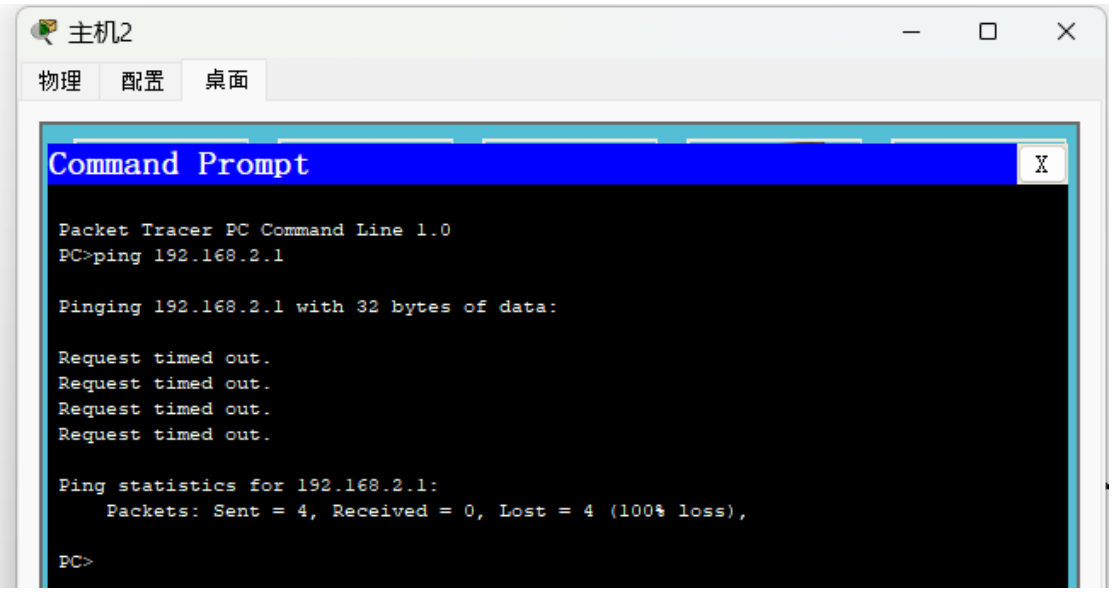
Request timed out.
Request timed out.
Request timed out.
Request timed out.

Ping statistics for 192.168.1.3:
    Packets: Sent = 4, Received = 0, Lost = 4 (100% loss),
```

PC0 到 PC3：



PC2 到 PC3：



第三章 网络程序设计系统分析

题目 5：基于 POP3 和 SMTP 的邮件客户端系统（建议 3 人组）

- 具有图形界面
- 可利用 SMTP 协议发送邮件
- 可利用 POP3 协议接受邮件

项目要求

- 基本功能
- 多线程（加分项）
- 图形界面

1. 需求分析

(1)功能需求
本邮件客户端系统需要实现用户注册登录、邮件服务器的配置管理、邮件的撰写发送、邮件的接收查看以及邮件的基本管理功能。具体包括用户通过邮箱密码创建账户并安全登录，配置 SMTP 服务器用于发送邮件，配置 POP3 服务器用于接收邮件。用户能够撰写包含文本格式和附件的邮件，发送给单个或多个收件人，并支持抄送和密送功能。系统需要自动从配置的 POP3 服务器接收新邮件，并在收件箱中展示邮件列表，用户可以查看邮件详情、回复转发邮件、删除邮件以及将重要邮件标记为星标。同时系统需要提供草稿箱保存未完成的邮件，已发送邮件归档，垃圾箱暂存删除的邮件，并支持简单的邮件搜索功能。

(2)非功能需求
系统在性能上要求界面操作响应流畅，邮件列表加载和内容展示无明显延迟。在邮件收发等耗时操作中，系统应通过多线程技术保持用户界面的可操作性。系统需要具备良好的可用性，提供清晰直观的图形界面和符合用户习惯的操作流程，确保用户易于学习和使用。可靠性方面，系统应能稳定处理网络异常和服务器连接中断等情况，保证用户数据不丢失。安全性需确保用户密码的加密存储、通信过程的安全以及操作权限的验证。

(3)系统需求
系统采用前后端分离的架构。前端使用 Vue.js 框架构建单页面应用，搭配 Element Plus 组件库实现用户界面，负责所有用户交互和数据显示。后端使用 Node.js 和 Express 框架提供 RESTful API，处理用户认证、邮件发送（通过 node-mailer 连接 SMTP 服务器）、邮件接收（通过 pop3 协议客户端）以及数据持久化等业务逻辑。开发与运行环境需要 Node.js 16+、npm 包管理器、现代浏览器（如 Chrome 90+），并可能需要配置反向代理（如 Nginx）用于生产环境部署。

题目要求设计一个基于 POP3 和 SMTP 协议的邮件客户端系统的需求分析。系统需要开发一个具有图形界面的邮件管理工具，支持用户通过 SMTP 协议发送邮件，并通过 POP3 协议接收邮件。核心功能包括用户的注册登录、邮件服务器的配置管理、邮件的撰写发送和接收查看，以及邮件的组织管理如草稿箱、已发送和垃圾箱等。

系统要求实现基本的邮件操作功能，包括写邮件、收邮件、查看邮件详情、删除邮件和搜索邮件等。作为加分项，系统需要支持多线程处理，例如在后台同步接收邮件时不阻塞用户界面操作，提升用户体验和系统响应速度。

整体项目需要具备完整的用户交互流程，从账户创建到邮件收发，界面应直观友好，操作便捷，同时保证邮件数据的安全性和传输的稳定性。

2. 可行性分析

（1） 技术可行性
本项目技术可行。项目使用的 Vue.js 前端框架、Node.js 后端技术、SMTP/POP3 通信协议均为成熟稳定的技术栈，拥有丰富的社区支持和开发文档。邮件协议处理可使用现有的 nodemailer 和 pop3 等可靠库实现，避免了从零开发协议解析的复杂性。团队具备 Web 开发和网络编程基础，项目功能均在现有技术能力范围内，无技术瓶颈。

（2） 经济可行性
本项目经济可行。开发工具均为开源免费软件（Vue.js、Node.js、VS Code 等），无需购买商业许可。运行环境可在个人计算机或学校服务器上部署，硬件成本极低。开发团队为学生小组，人力成本为零。维护阶段仅需基础服务器资源，整体经济投入可控，适合作为课程设计项目。

（3） 操作可行性
本项目操作可行。系统设计为图形化界面，采用常见的邮箱客户端交互模式（如收件箱、写邮件、草稿箱等），符合普通用户的操作习惯。界面将使用直观的图标和清晰的文字提示，学习成本低。系统仅需用户配置一次邮箱服务器信息，后续操作简单直接，适合非技术人员使用。

（4） 法律和伦理可行性
本项目法律和伦理可行。项目遵循 GPL/MIT 等开源协议使用相关开发库，无知识产权风险。系统处理用户邮箱密码时将进行加密存储和传输，不收集或滥用用户隐私数据，符合数据安全的基本伦理要求。项目为教学实践用途，不涉及商业盈利，无法律合规风险。

3. 部分技术栈介绍

（1） 关于 SMTP 协议

SMTP（简单邮件传输协议）是发送电子邮件的核心协议，基于 TCP 的 25 端口。
其主要方法包括：
核心 SMTP 方法：
a. HELO/EHLO - 客户端向服务器标识自己，EHLO 扩展了更多功能
b. MAIL FROM - 指定发件人地址
c. RCPT TO - 指定收件人地址（可多次使用）
d. DATA - 开始传输邮件正文
e. QUIT - 结束会话
f. AUTH - 身份验证（用于安全连接）
g. STARTTLS - 将明文连接升级为加密连接

Node.js 中使用 SMTP：
Node.js 通常通过 Nodemailer 库简化 SMTP 操作：

```
const nodemailer = require('nodemailer');

// 创建 SMTP 传输器
const transporter = nodemailer.createTransport({
  host: 'smtp.example.com',
  port: 587,
  secure: false, // TLS
  auth: {
    user: 'your-email@example.com',
    pass: 'your-password'
  }
});

// 发送邮件
async function sendMail() {
  const info = await transporter.sendMail({
    from: '"发件人" <sender@example.com>',
    to: 'recipient@example.com',
    subject: '测试邮件',
    text: '纯文本内容',
    html: '<b>HTML 内容</b>'
  });
  console.log('邮件已发送:', info.messageId);
}
```

模块引入：

backend > src > services > JS smtpService.js > SMTPService >

```
1   const nodemailer = require('nodemailer');
```

邮件发送的逻辑实现：

backend > src > services > JS smtpService.js > SMTPService > sendMail > info

```
3   class SMTPService {
29    async sendMail(mailOptions) {
33      const defaultOptions = {
34        from: mailOptions.from || this.config.username,
35        to: mailOptions.to,
36        subject: mailOptions.subject,
37        text: mailOptions.contentType === 'text/plain' ? mailOptions.content :
38        html: mailOptions.contentType === 'text/html' ? mailOptions.content :
39        attachments: mailOptions.attachments || []
40      };
```

（2） 关于 POP3 协议

POP3 核心方法:

a. USER - 指定用户名
b. PASS - 提供密码认证
c. STAT - 返回邮箱状态（邮件数量、总大小）
d. LIST - 列出所有邮件或特定邮件的详细信息
e. RETR - 检索指定邮件编号的完整内容
f. DELE - 标记邮件为删除（在 QUIT 后生效）
g. NOOP - 无操作，用于保持连接
h. RSET - 重置会话，取消删除标记
i. QUIT - 结束会话并执行删除操作
j. TOP - 检索邮件头部和指定行数的正文
k. UIDL - 获取邮件的唯一标识符

Node.js 中对 POP3 协议的调用:

```
const pop3 = require('poplib');
const client = new pop3(110, 'pop3.server.com');
client.on('connect', () => {
  console.log('已连接到 POP3 服务器');
  client.login('username', 'password');
});
client.on('login', (status, rawData) => {
  if (status) {
    client.list(); // 列出所有邮件
  }
});
client.on('list', (status, msgcount, msgnumber, data, rawData) => {
  if (status) {
    // 获取第一封邮件
    client.retr(1);
  }
});
client.on('retr', (status, msgnumber, data, rawData) => {
  if (status) {
    console.log('邮件内容:', data);
    client.dele(msgnumber); // 标记删除
  }
});
client.on('dele', (status, msgnumber, data, rawData) => {
```

```
  client.quit(); // 结束会话
});
```

模块引入：

backend > src > services > JS pop3Service.js > POP3Service > fetchEmails > email

```
1   const Pop3Command = require('pop3');
```

模块调用：

backend > src > services > JS pop3Service.js > POP3Service > fetchEmails > email

```
3   class POP3Service {
49    async getMailList(client) {
50      return new Promise((resolve, reject) => {
51        client.command('LIST', (err, status, msg) => {
52          if (err) return reject(err);
53
54          // 解析邮件列表
55          const mails = msg.trim().split('\n').slice(1).map(line => {
56            const [index, size] = line.split(' ');
57            return { index: parseInt(index), size: parseInt(size) };
58          });
59
60          resolve(mails);
61        });
62      });
63    }
```

（3） 异步调用语法糖 async/await

邮件发送、查询，前端和后端的查询交互属于 I/O 密集型任务，需要依赖于 async/await 异步调用执行避免因为网络请求缓慢而导致前端卡顿。

以下是初始化数据库的异步调用函数：

```
  async initializeDatabase() {
    try {
      await sequelize.authenticate();
      console.log('数据库连接成功');

      // 同步数据库（开发环境）
      if (process.env.NODE_ENV === 'development') {
        await sequelize.sync({ alter: false });
        console.log('数据库同步成功');
      }
    } catch (error) {
      console.error('数据库连接失败:', error);
      process.exit(1);
    }
  }
```

async/await 本质是基于 Promise 的语法糖，它将异步代码转化为类似同步的书写方式，提升代码可读性。在数据库初始化示例中：

a. 非阻塞特性：await sequelize.authenticate() 并不会阻塞主线程，而是将数据库连接操作放入事件队列，期间 JavaScript 主线程可继续处理其他任务。

b. 错误处理优化：通过 try-catch 结构统一处理异步错误，替代了传统的 Promise 链式 .catch() 方法，使错误处理更直观。

c. 执行流程控制：await 确保数据库连接成功后，再执行后续的同步操作(sequelize.sync)，这种顺序执行逻辑清晰，避免了回调地狱。

在实际邮件系统开发中，这种模式特别适合处理数据库查询、网络请求等 I/O 操作，既能保证代码执行顺序，又充分利用了 Node.js 的事件驱动优势，避免因等待数据库响应而导致整个应用阻塞。

（4） 多线程编程的具体案例

SMTP/POP3 邮件服务器氛围 pop3_server.py 和 smtp_server.py 两个部分：



在 start.py 中，创建了 stmp_thread 和 pop3_thread 两个县城同时运行，以提高系统资源的利用率：

```
# 创建线程运行服务器
smtp_thread = threading.Thread(target=run_smtp_server, daemon=True)
pop3_thread = threading.Thread(target=run_pop3_server, daemon=True)

smtp_thread.start()
pop3_thread.start()
```

同时，在 start.sh 一键启动脚本中，配置了拉起两个 python 邮件模拟后端，和 node.js 后端、vue 前端的操作：

拉起邮件服务器：

```
# 启动邮件服务器
nohup $PYTHON_CMD "$MAILSERVER_DIR/start.py" >> "$MAILSERVER_LOG" 2>&1 &
MAILSERVER_PID=$!
echo $MAILSERVER_PID > "$MAILSERVER_PID_FILE"

echo "邮件服务器进程PID: $MAILSERVER_PID"
```

拉起 node.js 后端：

```
# 启动后端
echo -e "${GREEN}启动后端服务...${NC}"
nohup node "$BACKEND_DIR/src/server.js" >> "$BACKEND_LOG" 2>&1 &
BACKEND_PID=$!
echo $BACKEND_PID > "$BACKEND_PID_FILE"
```

拉起 vue 前端：

```
# 启动前端
echo -e "${GREEN}启动前端服务 (端口: $FRONTEND_PORT)...${NC}"
nohup npm run dev >> "$FRONTEND_LOG" 2>&1 &
FRONTEND_PID=$!
echo $FRONTEND_PID > "$FRONTEND_PID_FILE"

echo "前端进程PID: $FRONTEND_PID"
```

（5） Javascript 的多线程机制

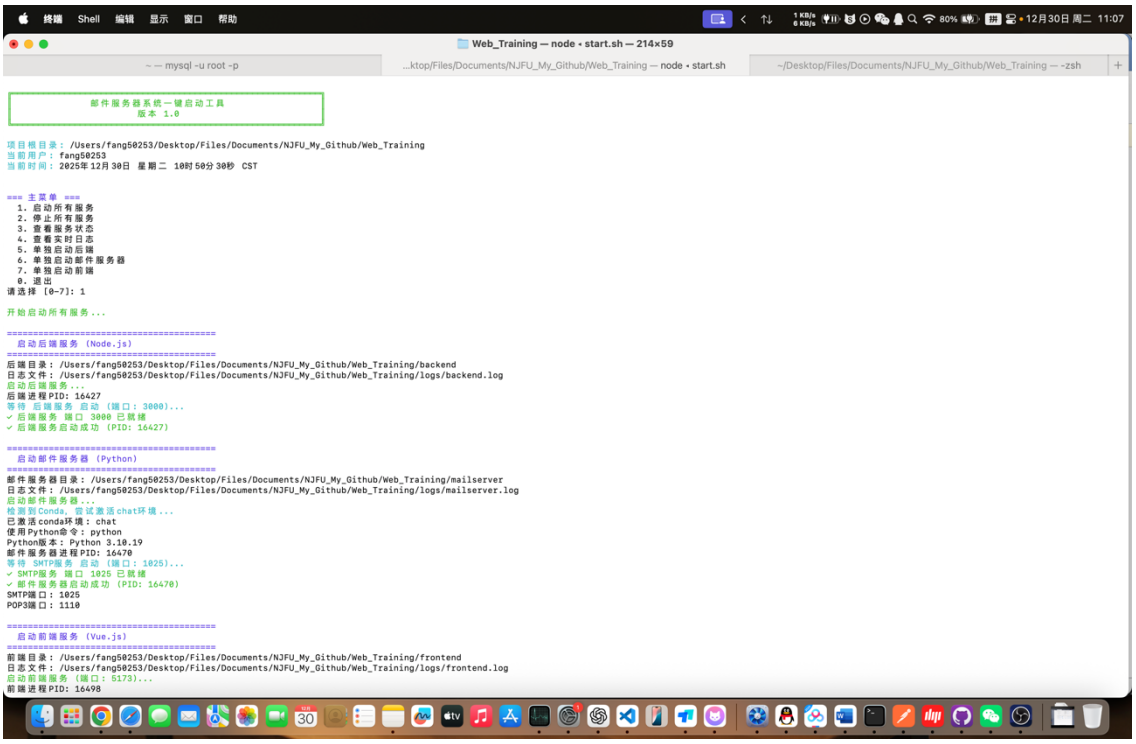
Javascript 本身是单线程的，但是可以通过 Worker 实现浏览器的 web 多线程，避免阻塞整个网页引发卡顿。

场景	方案	原因
UI 响应、网络请求	async/await	I/O 密集型，事件循环高效
图像处理、大数据计算	Worker Threads	CPU 密集型，避免阻塞主线程
实时数据同步	SharedArrayBuffer	需要线程间共享数据

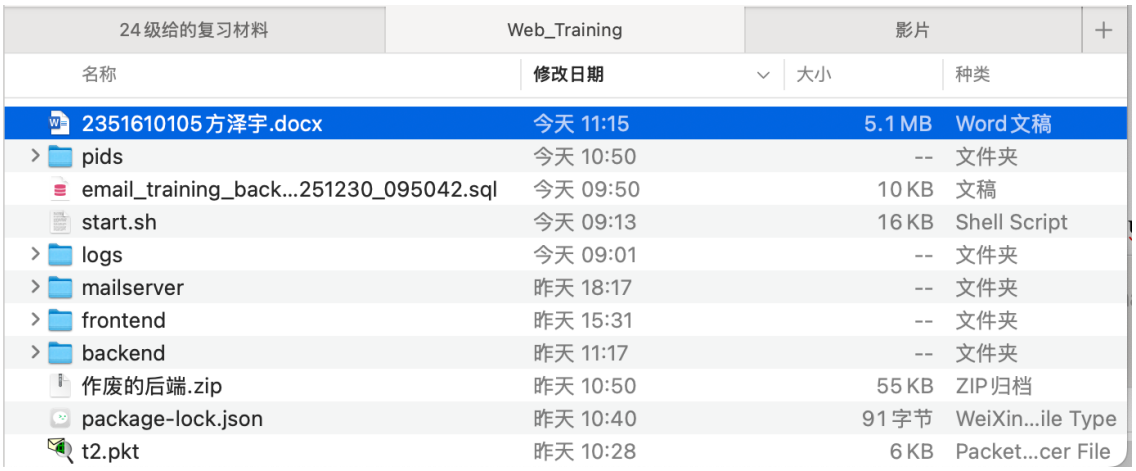
浏览器复杂计算	Web Workers	保持页面流畅

4. 系统界面设计

本系统配置了一键启动脚本 start.js，使得用户能够很方便地在 linux、unix 系系统上启动并使用它。

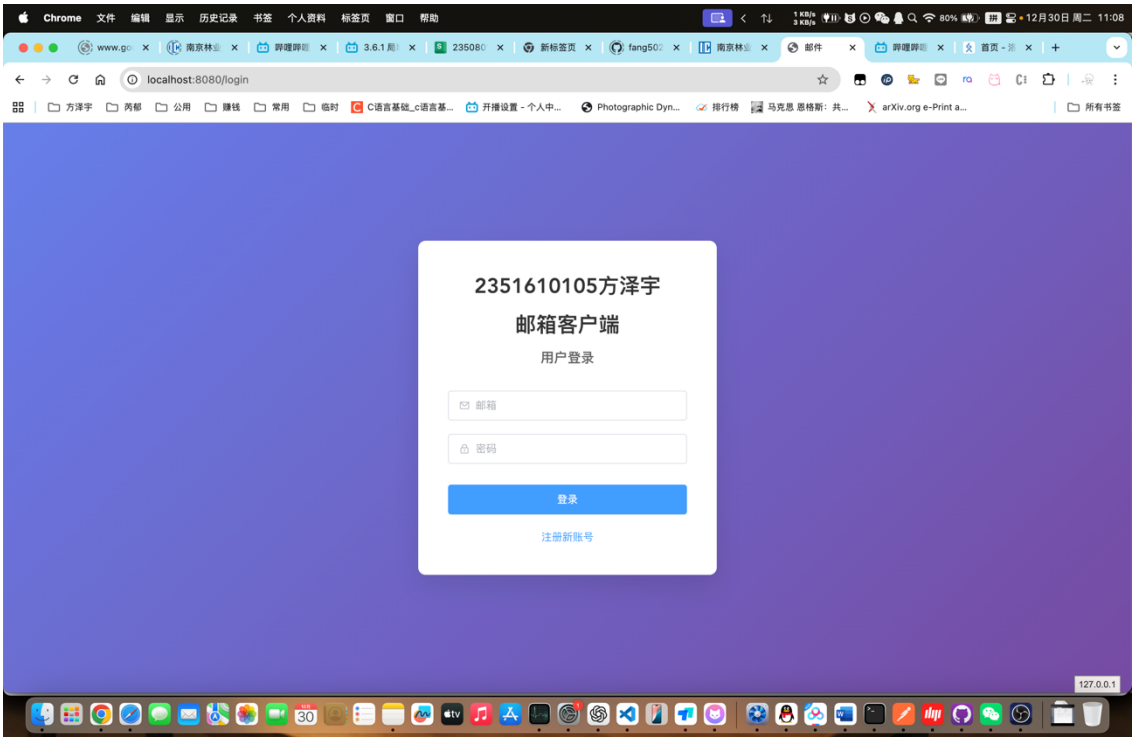


系统整体的文件结构：

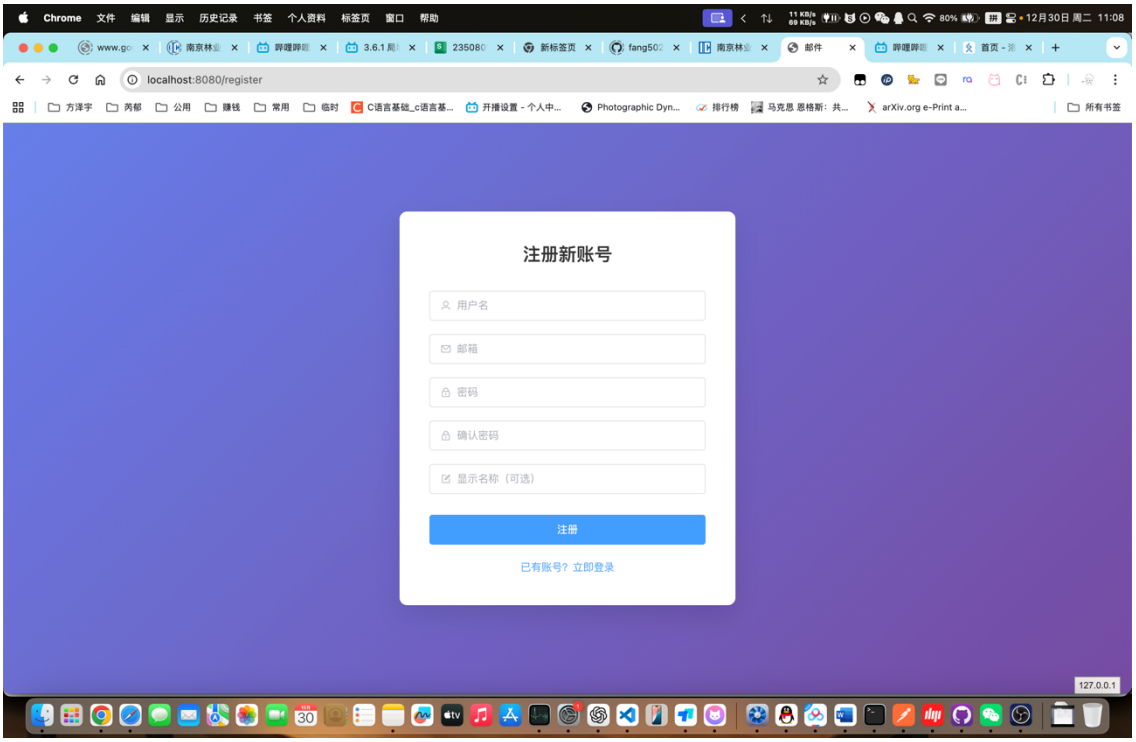


本系统整体采用单页应用架构，共设计了 6 个主要的视图页面来构建核心用户交互流程：

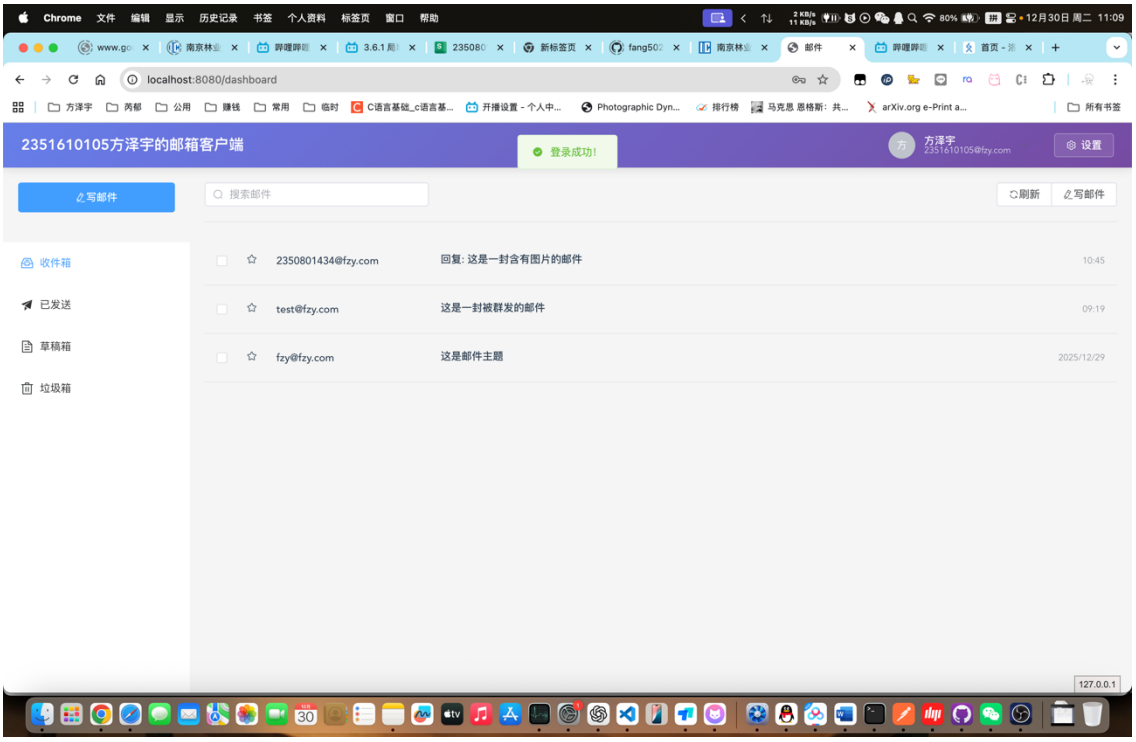
第一个是登录视图，提供用户身份验证入口，包含邮箱密码输入、表单验证和登录状态反馈。



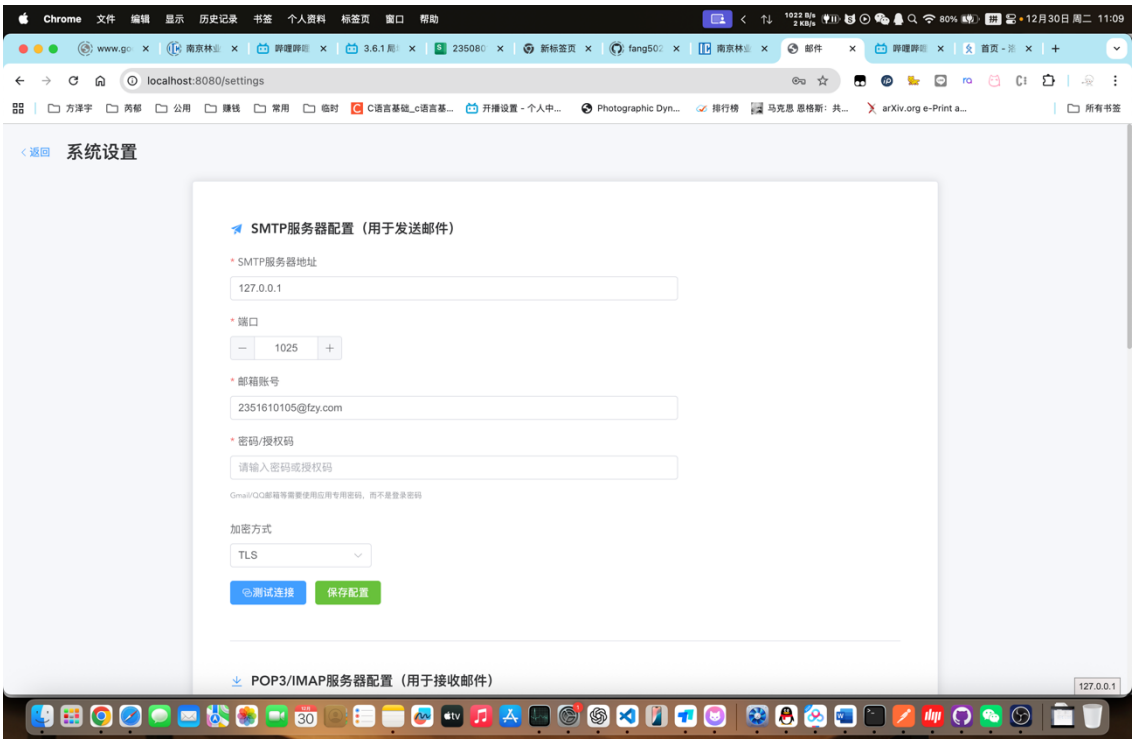
第二个是注册视图，处理新用户账户创建，包括信息填写、密码确认和注册结果提示。



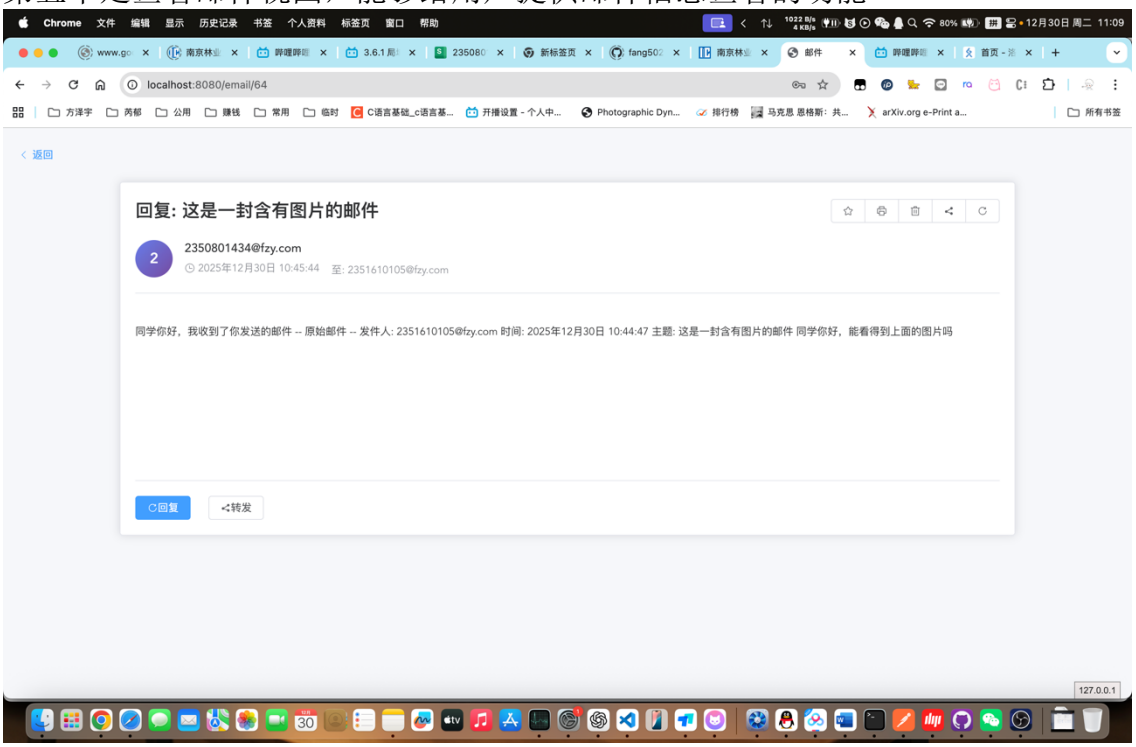
第三个是仪表盘视图，作为系统的主工作界面，集成了邮件列表展示、导航菜单和内容操作区域。



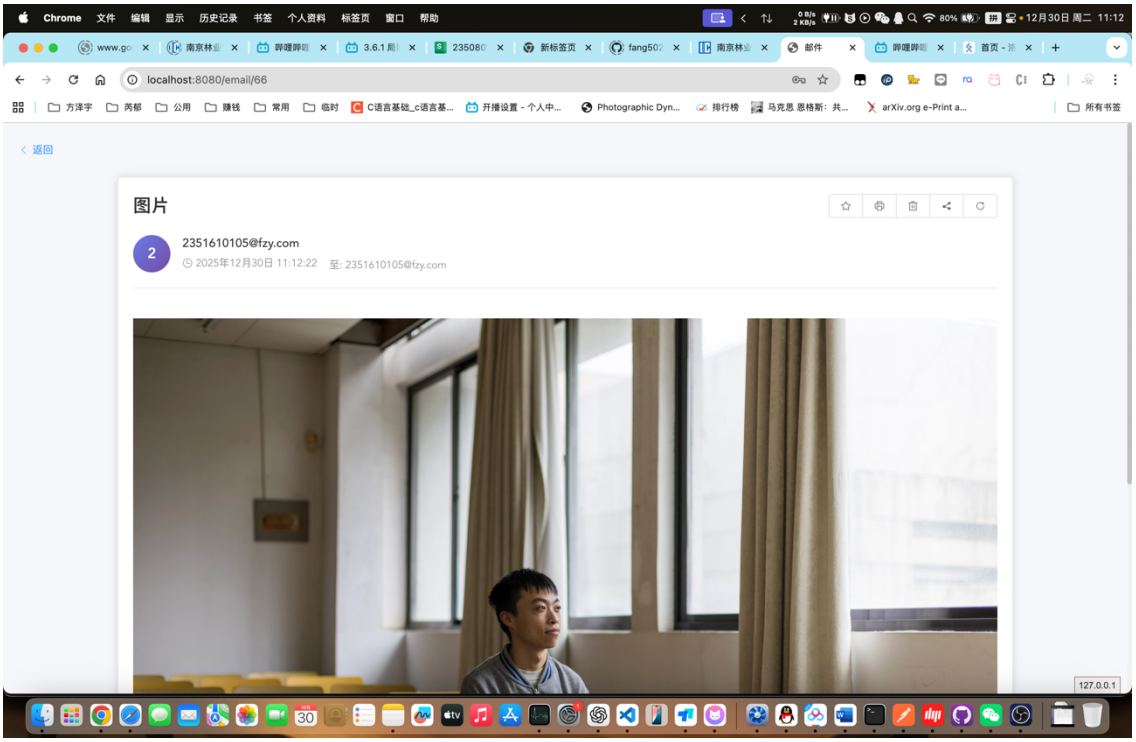
第四个是设置视图，专门用于邮箱服务器参数配置和用户偏好管理。



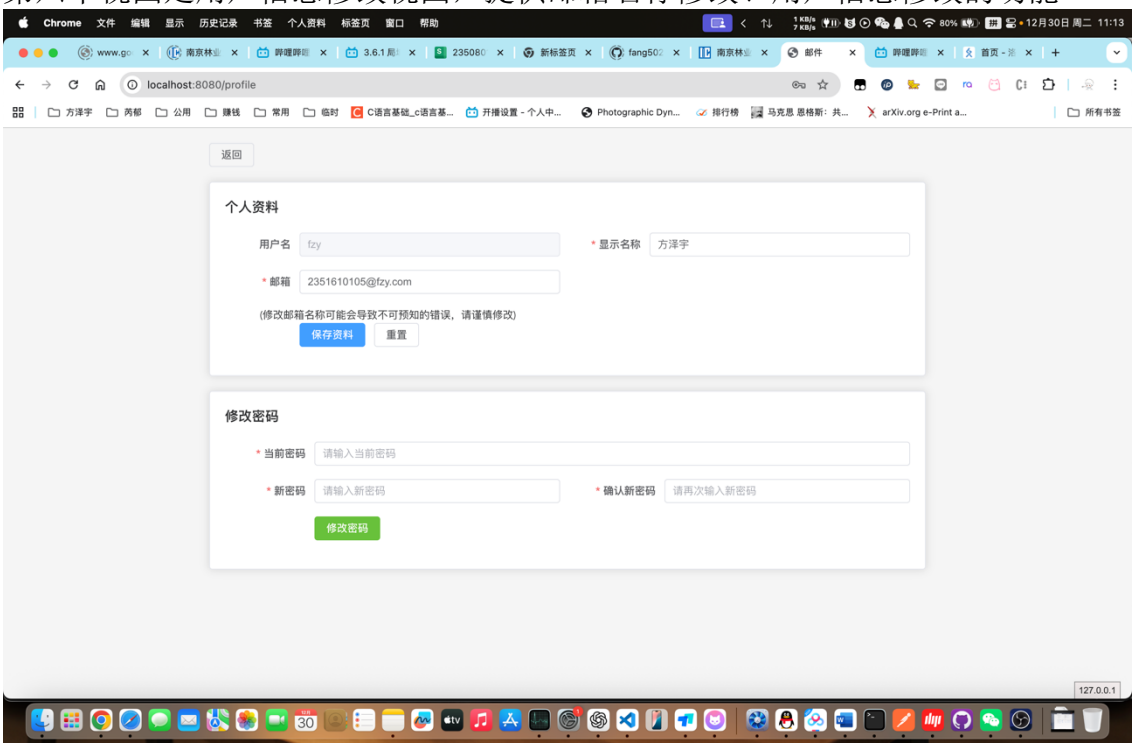
第五个是查看邮件视图，能够给用户提供邮件信息查看的功能



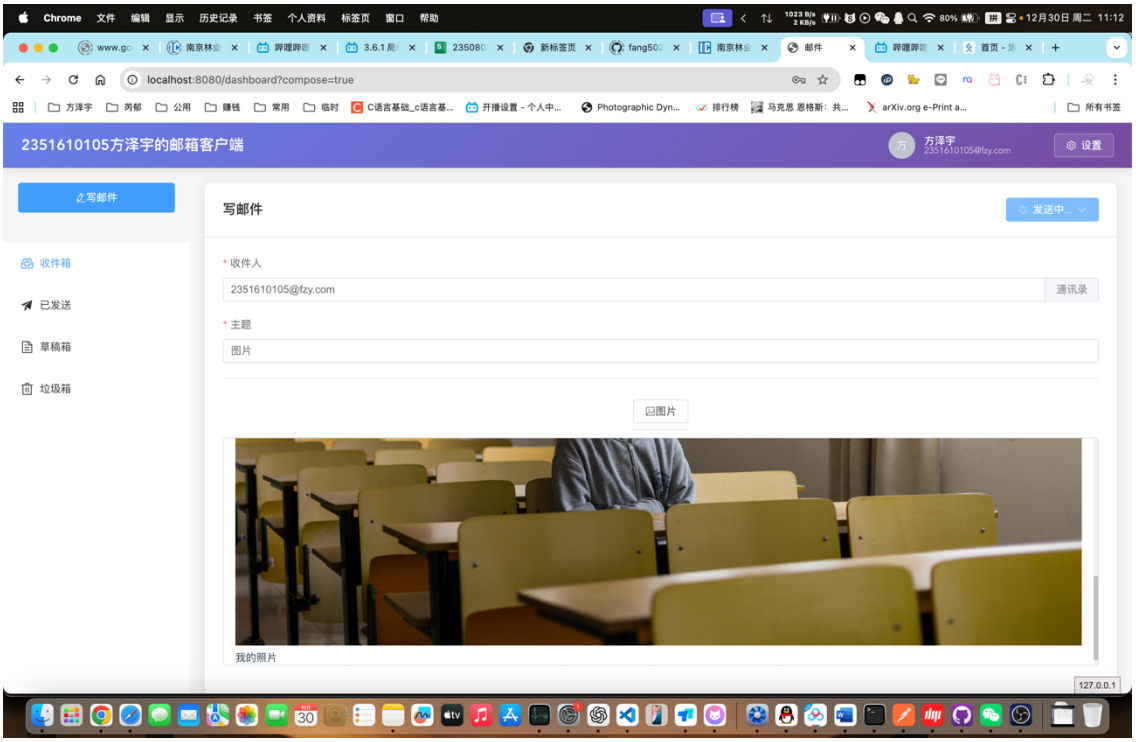
同时也提供了对图片邮件的查看



第六个视图是用户信息修改视图，提供邮箱名称修改、用户信息修改的功能



在组件化结构方面，系统实现了 2 个核心功能组件来支撑具体的业务操作：邮件操作组件负责邮件的编辑、发送和转发功能，集成了文本编辑器和图片插入的功能。



系统提供了邮件列表显示组件，能够从通过 smtp 协议从邮件服务器拉取邮件列表。



5. 开发工具介绍

（1） 系统环境
macOS 15.7.2 (24G325)，Apple M2Max 32GB

（2） Visual Studio Code1.107.1 (Universal)
集成开发环境使用 Visual Studio Code 1.107.1（Universal 版），其丰富的插件生态系统为前后端开发提供了代码提示、调试和版本控制等支持。

（3） Google Chrome143.0.7499.170 (正式版本) (arm64)
前端应用在 Google Chrome 143.0.7499.170（arm64 正式版）浏览器中进行调试和测试，确保在主流浏览器环境下的兼容性与性能表现。

（4） python3.10
后端邮件服务器模拟部分采用 Python 3.10 编写，其中安装的主要包如下：

邮件服务器基于 Flask 2.3.3 框架构建，提供 RESTful API 接口服务。数据库操作使用 SQLAlchemy 2.0.44 ORM 框架，通过 PyMySQL 1.1.0 驱动连接 MySQL 数据库，实现数据持久化存储。

用户认证和安全模块整合了多个专业库：Flask-JWT-Extended 4.5.2 负责 JSON Web Token 的生成与验证，Flask-Bcrypt 4.0.1 提供密码加密存储，cryptography 46.0.3 确保数据传输安全。PyJWT 2.10.1 辅助 JWT 令牌处理，bcrypt 4.0.1 实现高强度密码哈希。

网络与工具支持方面，requests 2.32.5 处理外部 HTTP 请求，python-dotenv 1.0.0 管理环境变量配置，Flask-Cors 4.0.0 解决跨域资源共享问题。这套库组合构建了稳定可靠的后端服务基础架构。

库名称	版本	主要功能	在项目中的用途
Flask	2.3.3	Python Web 框架	构建后端 API 服务器，提供 RESTful 接口
Flask-JWT-Extended	4.5.2	JWT 身份验证扩展	实现用户登录认证，生成和验证访问令牌
Flask-Bcrypt	4.0.1	密码加密扩展	对用户密码进行哈希加密存储

Flask-Cors	4.0.0	跨域资源共享扩展	允许前端应用跨域访问后端 API
SQLAlchemy	2.0.44	ORM 数据库工具	数据库对象关系映射，简化数据库操作
PyMySQL	1.1.0	MySQL 数据库驱动	连接和操作 MySQL 数据库
cryptography	46.0.3	加密算法库	提供安全的数据加密和解密功能
PyJWT	2.10.1	JSON Web Token 库	生成和解析 JWT 令牌
requests	2.32.5	HTTP 请求库	发送 HTTP 请求，与其他服务通信
python-dotenv	1.0.0	环境变量管理	从.env 文件加载配置信息
Flask-SQLAlchemy	3.0.5	Flask 集成 SQLAlchemy	在 Flask 应用中集成 SQLAlchemy

（5） anaconda 24.11.3
通过 Anaconda 24.11.3 环境进行依赖管理和虚拟环境隔离，保障了邮件协议服务的稳定运行。

（6） node 22.17.0
整个 Web 应用基于 Node.js 22.17.0 运行时环境构建，前端开发框架和后端 API 服务均在此版本上运行，确保了 JavaScript 特性的一致性和系统组件的兼容性。

后端：

库名称	版本	主要功能	在项目中的用途
express	^5.2.1	Node.js Web 框架	构建后端 API 服务器，提供 RESTful 接口服务
nodemailer	^7.0.12	SMTP 邮件发送库	实现邮件的发送功能，连接 SMTP 服务器发送邮件
pop3	^0.2.0	POP3 协议客户端库	连接 POP3 服务器接收邮件，实现邮件收取功能
sequelize	^6.37.7	ORM 数据库工具	对象关系映射，简化 MySQL 数据库操作，支持模型定义和数据迁移
mysql2	^3.16.0	MySQL 数据库驱动	高性能 MySQL 客户端，连接和操作 MySQL 数据库
jsonwebtoken	^9.0.3	JWT 令牌库	生成和验证 JSON Web Token，实现用户身份认证和会话管理
bcryptjs	^3.0.3	密码哈希库	对用户密码进行安全哈希加密和验证，保护用户账户安全
cors	^2.8.5	跨域资源共享中间件	处理跨域请求，允许前端应用访问后端 API
dotenv	^17.2.3	环境变量管理	从.env 文件加载配置信息，管理敏感数据和环境配置
helmet	^8.1.0	安全头部中间件	设置 HTTP 安全头部，增强应用的安全性防护

mailparser	^3.9.1	邮件解析库	解析接收到的邮件内容，提取邮件正文、附件等信息

前端：

库名称	版本	类型	主要功能	在项目中的用途
vue	^3.5.26	运行时依赖	Vue.js 框架核心	构建响应式用户界面的基础框架，支持组件化开发
vue-router	^4.6.4	运行时依赖	Vue.js 路由管理	实现单页面应用路由导航，管理页面切换和权限控制
pinia	^2.0.0	运行时依赖	Vue 状态管理库	集中管理应用状态，处理用户数据、邮件列表等共享状态
element-plus	^2.3.8	运行时依赖	UI 组件库	提供丰富的预制 UI 组件，快速构建美观的用户界面
@element-plus/icons-vue	^2.3.2	运行时依赖	Element Plus 图标库	提供美观的图标资源，增强界面视觉效果
axios	^1.6.2	运行时依赖	HTTP 客户端库	发送 HTTP 请求，与后端 API 进行数据通信
dayjs	^1.11.9	运行时依赖	日期时间处理库	格式化邮件时间戳，处理日期显示和比较
@vue/cli-service	~5.0.8	开发依赖	Vue CLI 构建工具	提供开发服务器、打包构建等工程化支持
@vue/cli-plugin-router	~5.0.8	开发依赖	Vue 路由插件	集成路由功能到 Vue 项目，简化路由配置
@vue/compiler-sfc	^3.2.13	开发依赖	Vue 单文件组件编译器	编译 Vue 单文件组件，支持模板和脚本处理

第四章 总结

经过一周的的网络工程和组网实习开发，我成功完成了一个基于 SMTP 和 POP3 协议的邮件客户端系统。本项目以 Vue.js 和 Element Plus 构建了现代化的前端界面，通过 Node.js 和 Express 实现了完整的后端 API 服务，同时开发了基于 Python Flask 的邮件服务器模拟环境。系统实现了用户注册登录、邮件服务器配置、邮件发送与接收、邮件管理等核心功能，并采用了多线程技术优化用户体验。

在技术实现层面，项目前端采用单页面应用架构，通过组件化开发提高了代码复用性和可维护性；后端实现了 RESTful API 设计，使用 JWT 进行用户身份认证；邮件处理模块集成了标准的 SMTP 和 POP3 协议通信。整个系统遵循模块化设计原则，各组件之间耦合度低，便于后续功能扩展和维护。

通过本次项目实践，我们不仅掌握了邮件协议的核心原理，还深入理解了 Web 全栈开发的技术栈整合方法，将课堂理论知识转化为实际可运行的系统，达到了课程设计的预期目标。

第五章 工作心得与体会

在这次邮件客户端系统的开发过程中，我们获得了宝贵的技术实践经验和团队协作体会。从最初的方案设计到最终的系统实现，每个阶段都让我们对软件开发全流程有了更深刻的理解。

技术层面上，我们体会到了协议理解的重要性。起初对 SMTP 和 POP3 协议的细节理解不够深入，导致在实现邮件发送和接收功能时遇到了诸多困难。通过查阅 RFC 文档、分析协议通信流程，我们逐步掌握了邮件协议的核心机制，这种从理论到实践的学习过程让我们对网络协议有了更直观的认识。

团队协作方面，我们学会了如何合理分工和有效沟通。前端、后端和邮件服务器的开发需要紧密配合，我们通过定期的代码评审、接口联调和问题讨论，确保各模块能够顺利集成。特别是在处理前后端数据交互时，我们深刻体会到接口设计规范化和文档完整性的重要性。

项目开发中最大的收获是解决问题的能力提升。面对各种技术难题，我们学会了如何有效搜索资料、分析问题根源、尝试不同解决方案。例如在实现附件上传功能时，我们先后尝试了多种 Base64 编码方案，最终选择了最符合邮件协议标准的方法。

这次课程设计让我们认识到，一个完整的软件系统不仅需要功能实现，更需要考虑用户体验、代码质量和系统可维护性。我们将把这些经验应用到未来的学习和工作中，不断提升自己的工程实践能力。


网络工程和组网实习
- 总 结 汇 报 P P T -
汇报人：2351610105-方泽宇

目录

CONTENTS

壹 组网部分
Phase work overview

贰 网络工程部分
Work completion

叁 项目成果展示
Project results display

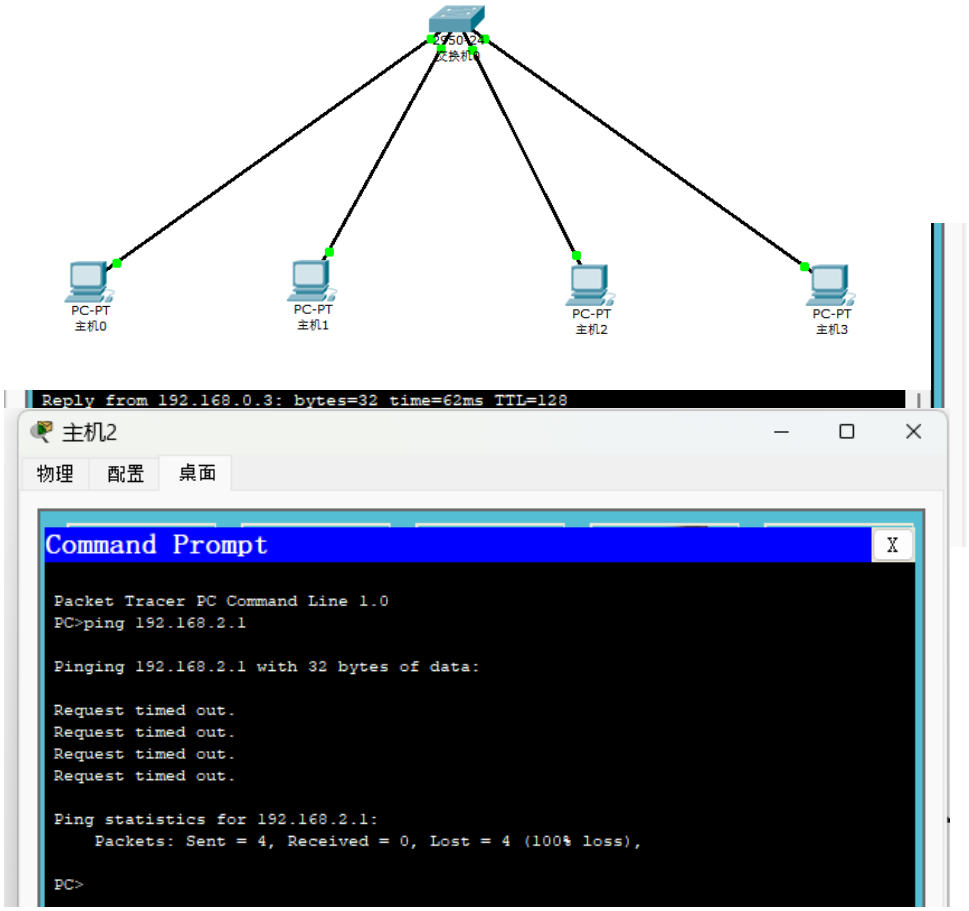
肆 未来工作计划
Future work plan

组网部分

-基于Packet Tracer组网配置-

Cisco Packet Tracer（思科数据包示踪器）是思科公司开发的一款强大的网络模拟器，主要面向学生和教师，用于学习、实验和练习网络配置与故障排除。它提供图形界面，允许用户通过拖拽方式搭建网络拓扑，模拟路由器、交换机等设备，并用类似真实IOS的命令进行配置，同时能可视化数据包在网络中的传输过程，非常适合CCNA等思科认证学习。

组 网 部 分



组 网 部 分

我通过Casio Packet Tracer配置了一个交换机的，掌握了二层交换机的基本配置和使用方法。通过Casio Packet Tracer进行了VLAN的配置，了解了虚拟网段下不同的子网无法访问，和子网掩码对局域网中设备的影响。


网络工程部分
-基于 POP3 和 SMTP 的邮件客户端系统-
具有图形界面
可利用SMTP协议发送邮件
可利用POP3协议接受邮件

网络工程部分

邮件服务

30%

使用python模拟了：

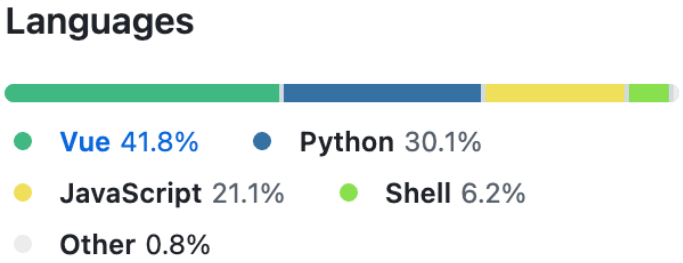
1.邮件发送服务器SMTP

2.邮件接收服务器POP3

后　　端

21%

使用nodejs框架，轻量化启动后端服务器，与POP3、SMTP服务器连接对数据库增删改查



前　　端

41%

采用vue框架，动态网页刷新，提供美观的、交互式的页面

其　　他

08%

Mysql数据库作为数据存储

Shell编写启动脚本

Nginx作为守护进程

仓库链接： https://github.com/fang50253/Web_Training

网络工程部分

关于SMTP协议（类POST请求）

SMTP（简单邮件传输协议）是发送电子邮件的核心协议，基于TCP的25端口。其主要方法包括：

核心SMTP方法：

a.HELO/EHLO - 客户端向服务器标识自己，EHLO扩展了更多功能

b.MAIL FROM - 指定发件人地址

c.RCPT TO - 指定收件人地址（可多次使用）

d.DATA - 开始传输邮件正文

e.QUIT - 结束会话

f.AUTH - 身份验证（用于安全连接）

g.STARTTLS - 将明文连接升级为加密连接



网络工程部分



关于POP3协议（类GET请求）

POP3核心方法:

a.USER - 指定用户名
b.PASS - 提供密码认证
c.STAT - 返回邮箱状态（邮件数量、总大小）
d.LIST - 列出所有邮件或特定邮件的详细信息
e.RETR - 检索指定邮件编号的完整内容
f.DELE - 标记邮件为删除（在QUIT后生效）
g.NOOP - 无操作，用于保持连接
h.RSET - 重置会话，取消删除标记
i.QUIT - 结束会话并执行删除操作
j.TOP - 检索邮件头部和指定行数的正文
k.UIDL - 获取邮件的唯一标识符

项目成果展示



项目结构

分为mailserver、backend和frontend

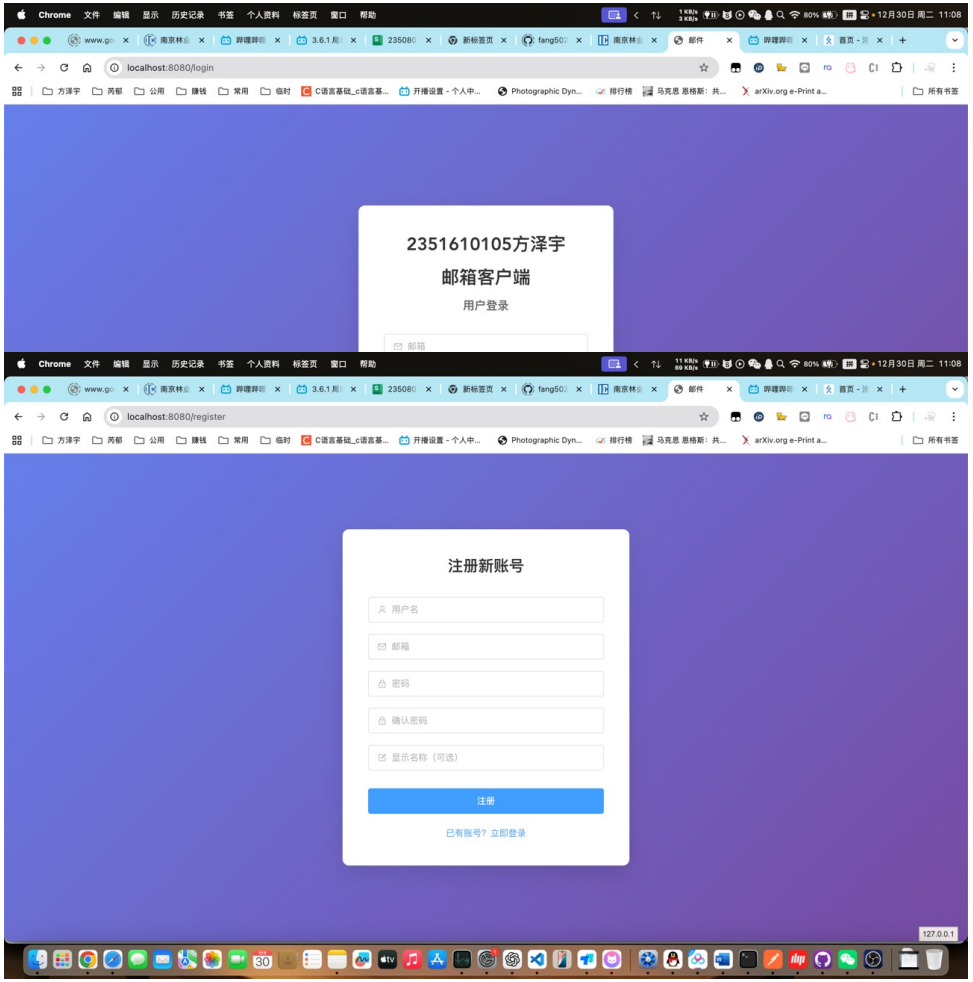
除mailserver因为轻量化实现，backend和frontend均采用MVC开发模式，虽然增加了代码量，但是降低了维护成本

项目成果展示

-基于 POP3 和 SMTP 的邮件客户端系统-

本节展示了项目成果

项目成果展示



登　　录

用户可以输入邮箱账号注册

用户名和邮箱账号均为唯一索引

注　　册

用户密码将会以hash值存储在数据库中

Jwt鉴权

使用jwt生成了expirt time为1d的jwt密钥，使得系统更加安全，除登录、注册外其他操作均需要在headers中携带jwt密钥

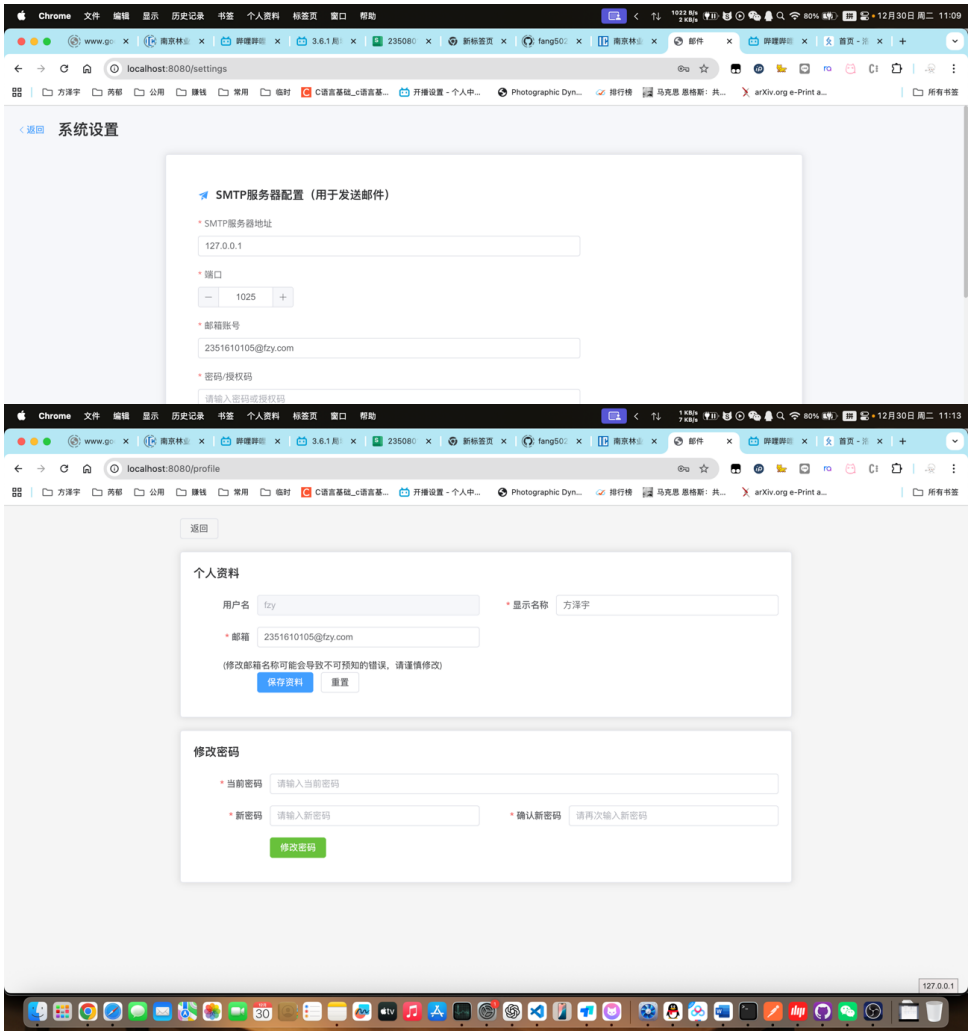
项目成果展示

邮箱服务器设置

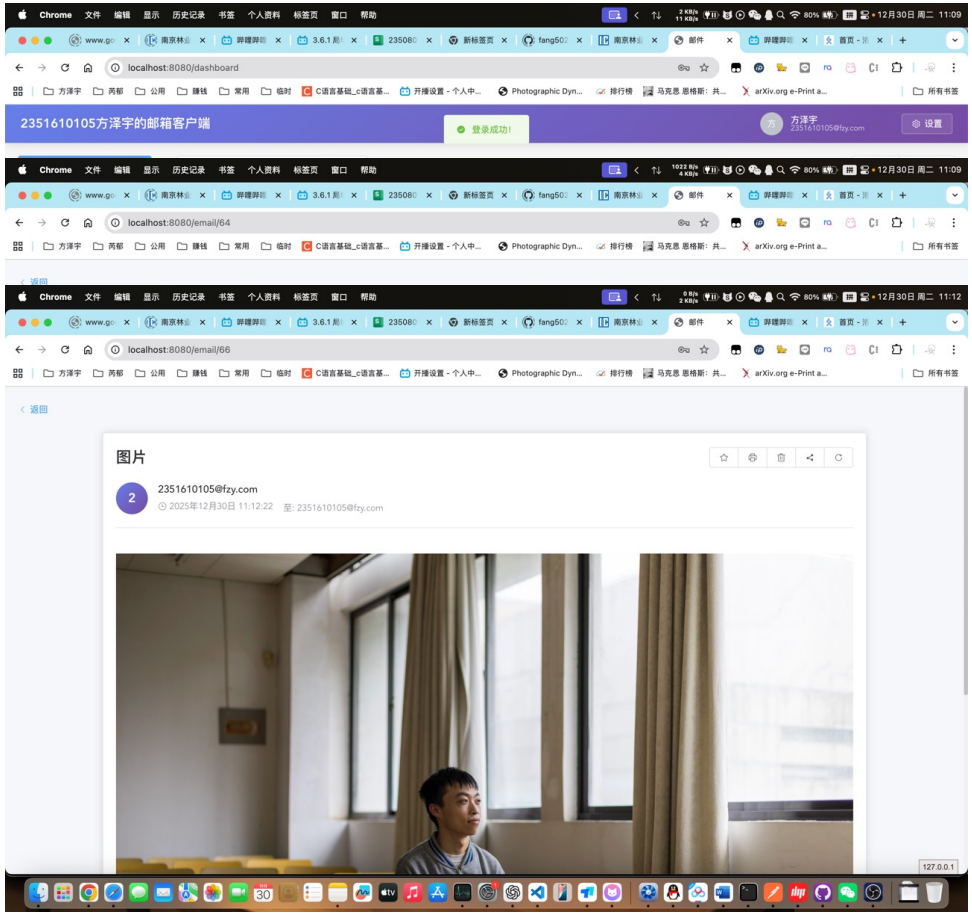
设置页面可以设置SMTP和POP3服务器，在本次实习中我们连接python虚拟邮箱客户端，因此填写localhost即可（因为macOS的25、995等端口无法使用，实习中采用其他端口实现）

个人资料设置

用户可以修改邮箱、显示名称和密码，用户名作为唯一主键，不可修改



项目成果展示



邮件拉取

前端用HTTP GET请求后端拉取邮件，后端调用POP3协议拉取邮件，数据库返回邮件后最终呈现在前端

邮件查看

用户单击即可看到邮件内容

含图邮件

邮件中的图片以base64编码存储在数据库中，趁现在用户面前


未来工作计划
-工作总结汇报PPT-
单击此处输入你的正文，文字是您思想的提炼，为了最终演示发布的良好效果，请尽量言简意赅的阐述观点；
根据需要可酌情增减文字，以便观者可以准确理解您所传达的信息。

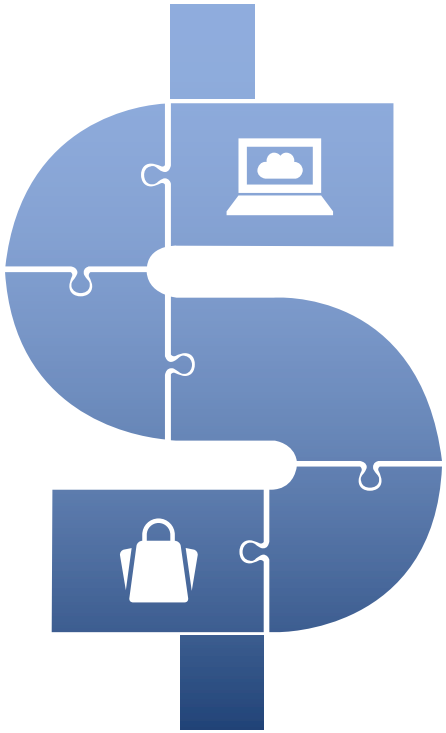
未来工作计划

新增附件支持

添加对象存储OOD，使得可以上传附件

部署到服务器

部署到相关linux远程服务器，申请工信部备案，开放真正的个人邮箱服务



开发手机客户端

使用微信小程序、SwiftUI等开发移动端

数据库迁移

使用Exchange Server等专业的邮箱数据库引擎，增强数据存储可靠性和对并发数据的处理能力


感谢您的观看
-工作总结汇报PPT-
单击此处输入你的正文，文字是您思想的提炼，为了最终演示发布的良好效果，请尽量言简意赅的阐述观点；
根据需要可酌情增减文字，以便观者可以准确理解您所传达的信息。
汇报人：稻小壳